



§. 关于动态内存申请后越界访问的深度讨论

★ 如何判断动态申请越界 (C方式, 注意源程序后缀为.c)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
int main()
{
```

```
    char *p;
    p = (char *)malloc(10 * sizeof(char));
    if (p == NULL)
        return -1;
```

```
    strcpy(p, "123456789");
```

① `p[10] = 'a';` //此句越界

`p[14] = 'A';` //此句越界

`p[15] = 'B';` //此句越界

② `p[10] = '\xfd';` //此句越界

```
    printf("addr:%p\n", p);
```

```
    for (int i = -4; i < 16; i++) //注意, 只有0-9是合理范围, 其余都是越界读
```

```
        printf("%p:%02x\n", (p+i), p[i]);
```

③ `free(p);`

```
    return 0;
```

```
}
```

在VS2022的x86/Debug模式下运行:

1、①②③全部注释, 观察运行结果

2、①放开, ②③注释, 观察运行结果

3、①③放开, ②注释, 观察运行结果

4、①②③全部放开, 观察运行结果

结论: VS的Debug模式是如何判断
动态申请内存访问越界的?

再观察下面四种环境下的运行结果:

VS2022 x86/Release

Dev 32bit-Debug

Dev 32bit-Release

Linux

每种讨论的结果可截图+文字说明,
如果几种环境的结果一致, 用一个
环境的截图+文字说明即可(可加页)

VS2022 x86/Debug



1、①②③全部注释

2、①放开，②③注释

3、①③放开，②注释

4、①②③全部放开

```
Microsoft Visual Studio 调试控制台
addr:006ABF40
006ABF3C:fffffffd
006ABF3D:fffffffd
006ABF3E:fffffffd
006ABF3F:fffffffd
006ABF40:31
006ABF41:32
006ABF42:33
006ABF43:34
006ABF44:35
006ABF45:36
006ABF46:37
006ABF47:38
006ABF48:39
006ABF49:00
006ABF4A:fffffffd
006ABF4B:fffffffd
006ABF4C:fffffffd
006ABF4D:fffffffd
006ABF4E:41
006ABF4F:42
```

```
Microsoft Visual Studio 调试控制台
addr:00D9B3B8
00D9B3B4:fffffffd
00D9B3B5:fffffffd
00D9B3B6:fffffffd
00D9B3B7:fffffffd
00D9B3B8:31
00D9B3B9:32
00D9B3BA:33
00D9B3BB:34
00D9B3BC:35
00D9B3BD:36
00D9B3BE:37
00D9B3BF:38
00D9B3C0:39
00D9B3C1:00
00D9B3C2:61
00D9B3C3:fffffffd
00D9B3C4:fffffffd
00D9B3C5:fffffffd
00D9B3C6:41
00D9B3C7:42
```

```
D:\desktop\oop\第六周（国庆跳过第五周）\week6\Debug\13-b7.exe
addr:00BB3B8
00BB3B4:fffffffd
00BB3B5:fffffffd
00BB3B6:fffffffd
00BB3B7:fffffffd
00BB3B8:31
00BB3B9:32
00BB3BA:33
00BB3BB:34
00BB3BC:35
00BB3BD:36
00BB3BE:37
00BB3BF:38
00BB3C0:39
00BB3C1:00
00BB3C2:61
00BB3C3:fffffffd
00BB3C4:fffffffd
00BB3C5:fffffffd
00BB3C6:41
00BB3C7:42
```

Microsoft Visual C++ Runtime Library

Debug Error!

Program: D:\desktop\oop\μÚÁúÖÜε"úÇìøýμÚİâÖÜε
©\week6\Debug\13-b7.exe

HEAP CORRUPTION DETECTED: after Normal block (#90) at 0x00BB3B8.
CRT detected that the application wrote to memory after end of heap
buffer.

(Press Retry to debug the application)

中止(A) 重试(R) 忽略(I)

```
Microsoft Visual Studio 调试控制台
addr:016BB3B8
016BB3B4:fffffffd
016BB3B5:fffffffd
016BB3B6:fffffffd
016BB3B7:fffffffd
016BB3B8:31
016BB3B9:32
016BB3BA:33
016BB3BB:34
016BB3BC:35
016BB3BD:36
016BB3BE:37
016BB3BF:38
016BB3C0:39
016BB3C1:00
016BB3C2:fffffffd
016BB3C3:fffffffd
016BB3C4:fffffffd
016BB3C5:fffffffd
016BB3C6:41
016BB3C7:42
```

结论：1.VS2022的x86/Debug模式下，只有自行释放动态申请的内存（即free语句）时才会判断内存访问是否越界。
2.该模式下，VS 会在动态分配的内存块的前后4个地址添加额外的特定值来判断访问是否越界。当这些值与特定值不符时，判断内存访问越界，报错；当值没有被改变，即使越界访问也不会判定错误。所以，当越界访问的位置超出特定值的范围（情况3），或者访问特定值的位置但并没有修改为不同的值时（情况2），程序并没有报错。



1、①②③全部注释

2、①放开，②③注释

3、①③放开，②注释

4、①②③全部放开

```
Microsoft Visual Studio 调试控制台
addr:006EBD40
006EBD38:ffffffd4
006EBD39:68
006EBD3A:61
006EBD3B:46
006EBD3C:00
006EBD3D:20
006EBD3E:00
006EBD3F:ffffff8e
006EBD40:31
006EBD41:32
006EBD42:33
006EBD43:34
006EBD44:35
006EBD45:36
006EBD46:37
006EBD47:38
006EBD48:39
006EBD49:00
006EBD4A:00
006EBD4B:00
006EBD4C:00
006EBD4D:00
006EBD4E:41
006EBD4F:42
```

```
Microsoft Visual Studio 调试控制台
addr:00CA9A90
00CA9A88:fffffffb
00CA9A89:48
00CA9A8A:70
00CA9A8B:ffffff9c
00CA9A8C:02
00CA9A8D:65
00CA9A8E:00
00CA9A8F:0e
00CA9A90:31
00CA9A91:32
00CA9A92:33
00CA9A93:34
00CA9A94:35
00CA9A95:36
00CA9A96:37
00CA9A97:38
00CA9A98:39
00CA9A99:00
00CA9A9A:61
00CA9A9B:00
00CA9A9C:00
00CA9A9D:00
00CA9A9E:41
00CA9A9F:42
```

```
Microsoft Visual Studio 调试控制台
addr:00789A90
00789A88:ffffffee
00789A89:ffffffbb
00789A8A:5c
00789A8B:fffffffe
00789A8C:ffffffc6
00789A8D:58
00789A8E:00
00789A8F:0e
00789A90:31
00789A91:32
00789A92:33
00789A93:34
00789A94:35
00789A95:36
00789A96:37
00789A97:38
00789A98:39
00789A99:00
00789A9A:61
00789A9B:00
00789A9C:00
00789A9D:00
00789A9E:41
00789A9F:42
```

```
Microsoft Visual Studio 调试控制台
addr:013A9A90
013A9A88:13
013A9A89:00
013A9A8A:ffffff95
013A9A8B:52
013A9A8C:64
013A9A8D:ffffff1
013A9A8E:00
013A9A8F:0e
013A9A90:31
013A9A91:32
013A9A92:33
013A9A93:34
013A9A94:35
013A9A95:36
013A9A96:37
013A9A97:38
013A9A98:39
013A9A99:00
013A9A9A:fffffffd
013A9A9B:00
013A9A9C:00
013A9A9D:00
013A9A9E:41
013A9A9F:42
```

结论：以上模式下，不会在动态分配的内存块的前后添加额外的特定值，不会因为内存访问越界报错。

Linux

1、①②③全部注释

```
[u2351520@oop ~]$ gcc -Wall
[u2351520@oop ~]$ ./13-b7c
addr:0x10f102a0
0x10f1029c:00
0x10f1029d:00
0x10f1029e:00
0x10f1029f:00
0x10f102a0:31
0x10f102a1:32
0x10f102a2:33
0x10f102a3:34
0x10f102a4:35
0x10f102a5:36
0x10f102a6:37
0x10f102a7:38
0x10f102a8:39
0x10f102a9:00
0x10f102aa:00
0x10f102ab:00
0x10f102ac:00
0x10f102ad:00
0x10f102ae:41
0x10f102af:42
[u2351520@oop ~]$ █
```

2、①放开，②③注释

```
[u2351520@oop ~]$ ./13-b7c
addr:0x32c802a0
0x32c8029c:00
0x32c8029d:00
0x32c8029e:00
0x32c8029f:00
0x32c802a0:31
0x32c802a1:32
0x32c802a2:33
0x32c802a3:34
0x32c802a4:35
0x32c802a5:36
0x32c802a6:37
0x32c802a7:38
0x32c802a8:39
0x32c802a9:00
0x32c802aa:61
0x32c802ab:00
0x32c802ac:00
0x32c802ad:00
0x32c802ae:41
0x32c802af:42
[u2351520@oop ~]$ █
```

3、①③放开，②注释

```
[u2351520@oop ~]$ gcc -Wall
[u2351520@oop ~]$ ./13-b7c
addr:0x24c702a0
0x24c7029c:00
0x24c7029d:00
0x24c7029e:00
0x24c7029f:00
0x24c702a0:31
0x24c702a1:32
0x24c702a2:33
0x24c702a3:34
0x24c702a4:35
0x24c702a5:36
0x24c702a6:37
0x24c702a7:38
0x24c702a8:39
0x24c702a9:00
0x24c702aa:61
0x24c702ab:00
0x24c702ac:00
0x24c702ad:00
0x24c702ae:41
0x24c702af:42
[u2351520@oop ~]$ █
```

4、①②③全部放开

```
[u2351520@oop ~]$ gcc -Wall
[u2351520@oop ~]$ ./13-b7c
addr:0x177502a0
0x1775029c:00
0x1775029d:00
0x1775029e:00
0x1775029f:00
0x177502a0:31
0x177502a1:32
0x177502a2:33
0x177502a3:34
0x177502a4:35
0x177502a5:36
0x177502a6:37
0x177502a7:38
0x177502a8:39
0x177502a9:00
0x177502aa:fd
0x177502ab:00
0x177502ac:00
0x177502ad:00
0x177502ae:41
0x177502af:42
[u2351520@oop ~]$ █
```



结论：Linux模式下，不会在动态分配的内存块的前后添加额外的特定值，不会因为内存访问越界报错（无论是否自行释放）。



§. 关于动态内存申请后越界访问的深度讨论

★ 如何判断动态申请越界 (C++方式, 注意源程序后缀为.cpp)

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
using namespace std;
```

```
int main()
{
```

```
    char *p;
    p = new(nothrow) char[10];
    if (p == NULL)
        return -1;
    strcpy(p, "123456789");
```

```
① p[10] = 'a';    //此句越界
   p[14] = 'A';    //此句越界
   p[15] = 'B';    //此句越界
```

```
② p[10] = '\xfd'; //此句越界
```

```
    cout << "addr:" << hex << (void *) (p) << endl;
    for (int i = -4; i < 16; i++) //注意, 只有0-9是合理范围, 其余都是越界读
        cout << hex << (void *) (p + i) << ":" << int(p[i]) << endl;
```

```
③ delete[] p;
```

```
    return 0;
```

```
}
```

在VS2022的x86/Debug模式下运行:

- 1、①②③全部注释, 观察运行结果
- 2、①放开, ②③注释, 观察运行结果
- 3、①③放开, ②注释, 观察运行结果
- 4、①②③全部放开, 观察运行结果

结论: VS的Debug模式是如何判断
动态申请内存访问越界的?

再观察下面四种环境下的运行结果:

VS2022 x86/Release

Dev 32bit-Debug

Dev 32bit-Release

Linux

每种讨论的结果可截图+文字说明,
如果几种环境的结果一致, 用一个
环境的截图+文字说明即可(可加页)

VS2022 x86/Debug



1、①②③全部注释

```
Microsoft Visual Studio 调试
addr:00CFC418
00CFC414:ffffffffd
00CFC415:ffffffffd
00CFC416:ffffffffd
00CFC417:ffffffffd
00CFC418:31
00CFC419:32
00CFC41A:33
00CFC41B:34
00CFC41C:35
00CFC41D:36
00CFC41E:37
00CFC41F:38
00CFC420:39
00CFC421:0
00CFC422:ffffffffd
00CFC423:ffffffffd
00CFC424:ffffffffd
00CFC425:ffffffffd
00CFC426:41
00CFC427:42
```

2、①放开，②③注释

```
Microsoft Visual Studio 调试
addr:00DD1998
00DD1994:ffffffffd
00DD1995:ffffffffd
00DD1996:ffffffffd
00DD1997:ffffffffd
00DD1998:31
00DD1999:32
00DD199A:33
00DD199B:34
00DD199C:35
00DD199D:36
00DD199E:37
00DD199F:38
00DD19A0:39
00DD19A1:0
00DD19A2:61
00DD19A3:ffffffffd
00DD19A4:ffffffffd
00DD19A5:ffffffffd
00DD19A6:41
00DD19A7:42
```

3、①③放开，②注释

```
D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7(C++.exe)
addr:01411B50
01411B4C:ffffffffd
01411B4D:ffffffffd
01411B4E:ffffffffd
01411B4F:ffffffffd
01411B50:31
01411B51:32
01411B52:33
01411B53:34
01411B54:35
01411B55:36
01411B56:37
01411B57:38
01411B58:39
01411B59:0
01411B5A:61
01411B5B:ffffffffd
01411B5C:ffffffffd
01411B5D:ffffffffd
01411B5E:41
01411B5F:42
```

Microsoft Visual C++ Runtime Library

Debug Error!

Program: ...sktop\oop\μÚÁúÖÜε"²úÇîle²ýμÚîâÖÜε
©\week6\Debug\13-b7(C++.exe)

HEAP CORRUPTION DETECTED: after Normal block (#167) at
0x01411B50.
CRT detected that the application wrote to memory after end of heap
buffer.

(Press Retry to debug the application)

中止(A) 重试(R) 忽略(I)

4、①②③全部放开

```
Microsoft Visual Studio 调试
addr:011216C0
011216BC:ffffffffd
011216BD:ffffffffd
011216BE:ffffffffd
011216BF:ffffffffd
011216C0:31
011216C1:32
011216C2:33
011216C3:34
011216C4:35
011216C5:36
011216C6:37
011216C7:38
011216C8:39
011216C9:0
011216CA:ffffffffd
011216CB:ffffffffd
011216CC:ffffffffd
011216CD:ffffffffd
011216CE:41
011216CF:42
```

结论: 1.VS2022的x86/Debug模式下, 只有自行释放动态申请的内存(即free语句)时才会判断内存访问是否越界。
2.该模式下, VS 会在动态分配的内存块的前后4个地址添加额外的特定值来判断访问是否越界。当这些值与特定值不符时, 判断内存访问越界, 报错; 当值没有被改变, 即使越界访问也不会判定错误。所以, 当越界访问的位置超出特定值的范围, 或者访问特定值的位置但并没有修改为不同的值时(情况2), 程序并没有报错。



1、①②③全部注释

```
Microsoft Visual Studio 调
addr:015ADF78
015ADF74:6c
015ADF75:0
015ADF76:0
015ADF77:ffffff8e
015ADF78:31
015ADF79:32
015ADF7A:33
015ADF7B:34
015ADF7C:35
015ADF7D:36
015ADF7E:37
015ADF7F:38
015ADF80:39
015ADF81:0
015ADF82:6f
015ADF83:0
015ADF84:6e
015ADF85:0
015ADF86:41
015ADF87:42
```

2、①放开，②③注释

```
Microsoft Visual Studio 调
addr:0143E0A0
0143E09C:4f
0143E09D:b
0143E09E:0
0143E09F:ffffff8e
0143E0A0:31
0143E0A1:32
0143E0A2:33
0143E0A3:34
0143E0A4:35
0143E0A5:36
0143E0A6:37
0143E0A7:38
0143E0A8:39
0143E0A9:0
0143E0AA:61
0143E0AB:0
0143E0AC:50
0143E0AD:0
0143E0AE:41
0143E0AF:42
```

3、①③放开，②注释

```
Microsoft Visual Studio 调
addr:014FE280
014FE27C:53
014FE27D:10
014FE27E:0
014FE27F:ffffff8e
014FE280:31
014FE281:32
014FE282:33
014FE283:34
014FE284:35
014FE285:36
014FE286:37
014FE287:38
014FE288:39
014FE289:0
014FE28A:61
014FE28B:0
014FE28C:49
014FE28D:0
014FE28E:41
014FE28F:42
```

4、①②③全部放开

```
Microsoft Visual Studio 调
addr:00CBE160
00CBE15C:3a
00CBE15D:4
00CBE15E:0
00CBE15F:ffffff8e
00CBE160:31
00CBE161:32
00CBE162:33
00CBE163:34
00CBE164:35
00CBE165:36
00CBE166:37
00CBE167:38
00CBE168:39
00CBE169:0
00CBE16A:fffffffd
00CBE16B:0
00CBE16C:6d
00CBE16D:0
00CBE16E:41
00CBE16F:42
```

结论：以上模式下，不会在动态分配的内存块的前后添加额外的特定值，不会因为内存访问越界报错。

Linux



1、①②③全部注释

```
[u2351520@oop ~]$ c++ -Wall  
[u2351520@oop ~]$ ./13-b7  
addr:0x1dc71eb0  
0x1dc71eac:0  
0x1dc71ead:0  
0x1dc71eae:0  
0x1dc71eaf:0  
0x1dc71eb0:31  
0x1dc71eb1:32  
0x1dc71eb2:33  
0x1dc71eb3:34  
0x1dc71eb4:35  
0x1dc71eb5:36  
0x1dc71eb6:37  
0x1dc71eb7:38  
0x1dc71eb8:39  
0x1dc71eb9:0  
0x1dc71eba:0  
0x1dc71ebb:0  
0x1dc71ebc:0  
0x1dc71ebd:0  
0x1dc71ebe:41  
0x1dc71ebf:42
```

2、①放开，②③注释

```
[u2351520@oop ~]$ c++ -Wall  
[u2351520@oop ~]$ ./13-b7  
addr:0x30b31eb0  
0x30b31eac:0  
0x30b31ead:0  
0x30b31eae:0  
0x30b31eaf:0  
0x30b31eb0:31  
0x30b31eb1:32  
0x30b31eb2:33  
0x30b31eb3:34  
0x30b31eb4:35  
0x30b31eb5:36  
0x30b31eb6:37  
0x30b31eb7:38  
0x30b31eb8:39  
0x30b31eb9:0  
0x30b31eba:61  
0x30b31ebb:0  
0x30b31ebc:0  
0x30b31ebd:0  
0x30b31ebe:41  
0x30b31ebf:42
```

3、①③放开，②注释

```
[u2351520@oop ~]$ c++ -Wall  
[u2351520@oop ~]$ ./13-b7  
addr:0x38ab1eb0  
0x38ab1eac:0  
0x38ab1ead:0  
0x38ab1eae:0  
0x38ab1eaf:0  
0x38ab1eb0:31  
0x38ab1eb1:32  
0x38ab1eb2:33  
0x38ab1eb3:34  
0x38ab1eb4:35  
0x38ab1eb5:36  
0x38ab1eb6:37  
0x38ab1eb7:38  
0x38ab1eb8:39  
0x38ab1eb9:0  
0x38ab1eba:61  
0x38ab1ebb:0  
0x38ab1ebc:0  
0x38ab1ebd:0  
0x38ab1ebe:41  
0x38ab1ebf:42
```

4、①②③全部放开

```
[u2351520@oop ~]$ c++ -Wall  
[u2351520@oop ~]$ ./13-b7  
addr:0x1cb51eb0  
0x1cb51eac:0  
0x1cb51ead:0  
0x1cb51eae:0  
0x1cb51eaf:0  
0x1cb51eb0:31  
0x1cb51eb1:32  
0x1cb51eb2:33  
0x1cb51eb3:34  
0x1cb51eb4:35  
0x1cb51eb5:36  
0x1cb51eb6:37  
0x1cb51eb7:38  
0x1cb51eb8:39  
0x1cb51eb9:0  
0x1cb51eba:fd  
0x1cb51ebb:0  
0x1cb51ebc:0  
0x1cb51ebd:0  
0x1cb51ebe:41  
0x1cb51ebf:42
```

结论：Linux模式下，不会在动态分配的内存块的前后添加额外的特定值，不会因为内存访问越界报错（无论是否自行释放）。



§. 关于动态内存申请后越界访问的深度讨论

★ 如何判断普通数组的越界访问（C方式，**注意源程序后缀为.c**）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char a[10] = "0123456789";

    /*①*/ a[10] = 'a'; //此句越界
    a[14] = 'A'; //此句越界
    a[15] = 'B'; //此句越界
    /*②*/ a[10] = '\xcc'; //此句越界
    printf("addr-a:%p\n", a);
    for (int i = -4; i < 16; i++) //注意，只有0-9是合理范围，其
        printf("%p:%02x\n", (a + i), a[i]);

    return 0;
}
```

在理解P. 1/P. 2的情况下，自行构造相似的程序，来观察数组越界后的内存表现，并验证与动态申请是否相似

要求：

- 1、数组用 `char a[10];` 形式
- 3、测试程序在下面五种环境下运行
VS2022 x86/Debug
VS2022 x86/Release
Dev 32bit-Debug
Dev 32bit-Release
Linux
- 4、每种讨论的结果可截图+文字说明，如果几种环境的结果一致，用一个环境的截图+文字说明即可(可加页)

VS2022 x86/Debug



1、①②全部注释

```
Microsoft Visual Studio 调试
addr:0075FB84
0075FB80:ffffffcc
0075FB81:ffffffcc
0075FB82:ffffffcc
0075FB83:ffffffcc
0075FB84:30
0075FB85:31
0075FB86:32
0075FB87:33
0075FB88:34
0075FB89:35
0075FB8A:36
0075FB8B:37
0075FB8C:38
0075FB8D:39
0075FB8E:ffffffcc
0075FB8F:ffffffcc
0075FB90:ffffffcc
0075FB91:ffffffcc
0075FB92:41
0075FB93:42
```

2、①放开，②注释

D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe

addr:00CFCC4
00CFCC0:ffffffcc
00CFCC1:ffffffcc
00CFCC2:ffffffcc
00CFCC3:ffffffcc
00CFCC4:30
00CFCC5:31
00CFCC6:32
00CFCC7:33
00CFCC8:34
00CFCC9:35
00CFCCA:36
00CFCCB:37
00CFCCC:38
00CFCCD:39
00CFCCF:ffffffcc
00CFCD0:ffffffcc
00CFCD1:ffffffcc
00CFCD2:41
00CFCD3:42

Microsoft Visual C++ Runtime Library

Debug Error!
Program: D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe
Module: D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe
File:
Run-Time Check Failure #2 - Stack around the variable 'a' was corrupted.
(Press Retry to debug the application)

中止(A) 重试(R) 忽略(I)

3、①②全部放开

```
Microsoft Visual Studio 调试
addr:008FF8E4
008FF8E0:ffffffcc
008FF8E1:ffffffcc
008FF8E2:ffffffcc
008FF8E3:ffffffcc
008FF8E4:30
008FF8E5:31
008FF8E6:32
008FF8E7:33
008FF8E8:34
008FF8E9:35
008FF8EA:36
008FF8EB:37
008FF8EC:38
008FF8ED:39
008FF8EE:ffffffcc
008FF8EF:ffffffcc
008FF8F0:ffffffcc
008FF8F1:ffffffcc
008FF8F2:41
008FF8F3:42
```

结论：1.VS2022的x86/Debug模式下，会判断数组内存访问是否越界。

2.该模式下，VS 会在动态分配的内存块的前后4个地址添加额外的特定值来判断访问是否越界。当这些值与特定值不符时，判断内存访问越界，报错；当值没有被改变，即使越界访问也不会判定错误。所以，当越界访问的位置超出特定值的范围，或者访问特定值的位置但并没有修改为不同的值时（情况3），程序并没有报错。

1、①②全部注释

```
Microsoft Visual Studio 调试
addr:00D7FB30
00D7FB2C:28
00D7FB2D:ffffffb2
00D7FB2E:2f
00D7FB2F:01
00D7FB30:30
00D7FB31:31
00D7FB32:32
00D7FB33:33
00D7FB34:34
00D7FB35:35
00D7FB36:36
00D7FB37:37
00D7FB38:38
00D7FB39:39
00D7FB3A:ffffffb9
00D7FB3B:76
00D7FB3C:03
00D7FB3D:13
00D7FB3E:ffffffd8
00D7FB3F:00
```

2、①放开，②注释

```
Microsoft Visual Studio
addr:00B3FBAC
00B3FBA8:28
00B3FBA9:ffffffb2
00B3FBAA:ffffffe3
00B3FBAB:00
00B3FBAC:30
00B3FBAD:31
00B3FBAE:32
00B3FBAF:33
00B3FBB0:34
00B3FBB1:35
00B3FBB2:36
00B3FBB3:37
00B3FBB4:38
00B3FBB5:39
00B3FBB6:61
00B3FBB7:76
00B3FBB8:05
00B3FBB9:13
00B3FBBA:05
00B3FBBB:00
```

3、①②全部放开

```
Microsoft Visual Studio
addr:00D7FBBC
00D7FBB8:28
00D7FBB9:ffffffb2
00D7FBBA:0f
00D7FBBB:01
00D7FBBC:30
00D7FBBD:31
00D7FBBE:32
00D7FBBF:33
00D7FBC0:34
00D7FBC1:35
00D7FBC2:36
00D7FBC3:37
00D7FBC4:38
00D7FBC5:39
00D7FBC6:ffffffcc
00D7FBC7:76
00D7FBC8:05
00D7FBC9:13
00D7FBCA:ffffff97
00D7FBCB:00
```

结论：该模式下，不会因为数组的内存访问越界报错，但似乎只能修改超过边界4个地址以内，修改其他更远位置是无效的。



1、①②全部注释

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffc0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

2、①放开，②注释

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffd0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

3、①②全部放开

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffd0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

结论：该模式下，不会因为数组的内存访问越界报错，但似乎只能修改超过边界4个地址以外，修改位置4个地址以内的值是无效的。

1、①②全部注释

2、①放开，②注释

3、①②放开

```
[u2351520@oop ~]$ c++ -Wa  
[u2351520@oop ~]$ ./13-b7  
addr: 0xfffffc99d480  
0xfffffc99d470:fc99d4b0  
0xfffffc99d474:ffff  
0xfffffc99d478:bc714384  
0xfffffc99d47c:ffff  
0xfffffc99d480:0  
0xfffffc99d484:1  
0xfffffc99d488:2  
0xfffffc99d48c:3  
0xfffffc99d490:4  
0xfffffc99d494:5  
0xfffffc99d498:6  
0xfffffc99d49c:7  
0xfffffc99d4a0:8  
0xfffffc99d4a4:9  
0xfffffc99d4a8:0  
0xfffffc99d4ac:b  
0xfffffc99d4b0:0  
0xfffffc99d4b4:0  
0xfffffc99d4b8:e  
0xfffffc99d4bc:f
```

```
[u2351520@oop ~]$ c++ -Wa  
[u2351520@oop ~]$ ./13-b7  
addr: 0xfffff7f9f430  
0xfffff7f9f420:f7f9f460  
0xfffff7f9f424:ffff  
0xfffff7f9f428:b9bd4384  
0xfffff7f9f42c:ffff  
0xfffff7f9f430:0  
0xfffff7f9f434:1  
0xfffff7f9f438:2  
0xfffff7f9f43c:3  
0xfffff7f9f440:4  
0xfffff7f9f444:5  
0xfffff7f9f448:6  
0xfffff7f9f44c:7  
0xfffff7f9f450:8  
0xfffff7f9f454:9  
0xfffff7f9f458:a  
0xfffff7f9f45c:b  
0xfffff7f9f460:0  
0xfffff7f9f464:0  
0xfffff7f9f468:e  
0xfffff7f9f46c:f
```

```
[u2351520@oop ~]$ ./13-b7  
addr: 0xffffe189b370  
0xffffe189b360:e189b3a0  
0xffffe189b364:ffff  
0xffffe189b368:a6b14384  
0xffffe189b36c:ffff  
0xffffe189b370:0  
0xffffe189b374:1  
0xffffe189b378:2  
0xffffe189b37c:3  
0xffffe189b380:4  
0xffffe189b384:5  
0xffffe189b388:6  
0xffffe189b38c:7  
0xffffe189b390:8  
0xffffe189b394:9  
0xffffe189b398:cccccccc  
0xffffe189b39c:b  
0xffffe189b3a0:0  
0xffffe189b3a4:0  
0xffffe189b3a8:e  
0xffffe189b3ac:f
```

结论：Linux模式下，不会因为数组的内存访问越界报错。



§. 关于动态内存申请后越界访问的深度讨论

★ 如何判断普通数组的越界访问（C方式，**注意源程序后缀为.c**）

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
int main()
```

```
{
```

```
    int a[10] = { 0,1,2,3,4,5,6,7,8,9 };
```

```
    /*①*/ a[10] = 10; //此句越界
```

```
    a[14] = 14; //此句越界
```

```
    a[15] = 15; //此句越界
```

```
    /*②*/ a[10] = 0xc0000000; //此句越界
```

```
    printf("addr:%p\n", a);
```

```
    for (int i = -4; i < 16; i++) //注意，只有0-9是合理范围，其  
    余都是越界读
```

```
        printf("%p:%02x\n", (a + i), a[i]);
```

```
    return 0;
```

```
}
```

在理解P. 1/P. 2的情况下，自行构造相似的程序，来观察数组越界后的内存表现，并验证与动态申请是否相似

要求：

2、数组用 `int a[10];` 形式

3、测试程序在下面五种环境下运行

VS2022 x86/Debug

VS2022 x86/Release

Dev 32bit-Debug

Dev 32bit-Release

Linux

4、每种讨论的结果可截图+文字说明，如果几种环境的结果一致，用一个环境的截图+文字说明即可（可加页）

VS2022 x86/Debug



1、①②全部注释

```
Microsoft Visual Studio 调试
addr:0052FB6C
0052FB5C:cccccccc
0052FB60:ffffffffd
0052FB64:cccccccc
0052FB68:cccccccc
0052FB6C:00
0052FB70:01
0052FB74:02
0052FB78:03
0052FB7C:04
0052FB80:05
0052FB84:06
0052FB88:07
0052FB8C:08
0052FB90:09
0052FB94:cccccccc
0052FB98:7e1173e1
0052FB9C:52fbbc
0052FBA0:5d2083
0052FBA4:0e
0052FBA8:0f
```

2、①放开，②注释

```
D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe
addr:007DFD3C
007DFD2C:cccccccc
007DFD30:ffffffffd
007DFD34:cccccccc
007DFD38:cccccccc
007DFD3C:00
007DFD40:01
007DFD44:02
007DFD48:03
007DFD4C:04
007DFD50:05
007DFD54:06
007DFD58:07
007DFD5C:08
007DFD60:09
007DFD64:0a
007DFD68:e927f944
007DFD6C:7dfd8c
007DFD70:4c2083
007DFD74:0e
007DFD78:0f
```

Microsoft Visual C++ Runtime Library

Debug Error!

Program: D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe
Module: D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe
File:

Run-Time Check Failure #2 - Stack around the variable 'a' was corrupted.

(Press Retry to debug the application)

中止(A) 重试(R) 忽略(I)

3、①②全部放开

```
Microsoft Visual Studio 调试
addr:004FFC78
004FFC68:cccccccc
004FFC6C:ffffffffd
004FFC70:cccccccc
004FFC74:cccccccc
004FFC78:00
004FFC7C:01
004FFC80:02
004FFC84:03
004FFC88:04
004FFC8C:05
004FFC90:06
004FFC94:07
004FFC98:08
004FFC9C:09
004FFCA0:cccccccc
004FFCA4:817f195f
004FFCA8:4ffcc8
004FFCAC:e62083
004FFCB0:0e
004FFCB4:0f
```

结论：1.VS2022的x86/Debug模式下，会判断数组内存访问是否越界。

2.该模式下，VS 会在动态分配的内存块的前后4个地址添加额外的特定值来判断访问是否越界。当这些值与特定值不符时，判断内存访问越界，报错；当值没有被改变，即使越界访问也不会判定错误。所以，当越界访问的位置超出特定值的范围，或者访问特定值的位置但并没有修改为不同的值时（情况3），程序并没有报错。

1、①②全部注释

```
Microsoft Visual Studio 调试
addr:0135F8FC
0135F8EC:682108
0135F8F0:682108
0135F8F4:15c04e8
0135F8F8:15bb228
0135F8FC:00
0135F900:01
0135F904:02
0135F908:03
0135F90C:04
0135F910:05
0135F914:06
0135F918:07
0135F91C:08
0135F920:09
0135F924:703cbe59
0135F928:135f970
0135F92C:681289
0135F930:01
0135F934:15bb228
0135F938:15c04e8
```

2、①放开，②注释

```
Microsoft Visual Studio 调试
addr:00F8FC68
00F8FC58:2e2108
00F8FC5C:2e2108
00F8FC60:12c04e8
00F8FC64:12bb228
00F8FC68:00
00F8FC6C:01
00F8FC70:02
00F8FC74:03
00F8FC78:04
00F8FC7C:05
00F8FC80:06
00F8FC84:07
00F8FC88:08
00F8FC8C:09
00F8FC90:0a
00F8FC94:c6e2138a
00F8FC98:f8fce0
00F8FC9C:2e1297
00F8FCA0:01
00F8FCA4:12bb228
```

3、①②全部放开

```
Microsoft Visual Studio 调试
addr:007BF8A4
007BF894:672108
007BF898:672108
007BF89C:a304e8
007BF8A0:a2b228
007BF8A4:00
007BF8A8:01
007BF8AC:02
007BF8B0:03
007BF8B4:04
007BF8B8:05
007BF8BC:06
007BF8C0:07
007BF8C4:08
007BF8C8:09
007BF8CC:cccccccc
007BF8D0:1f32328d
007BF8D4:7bf91c
007BF8D8:671297
007BF8DC:01
007BF8E0:a2b228
```

结论：该模式下，不会因为数组的内存访问越界报错，但似乎只能修改超过边界4个地址以内，修改其他更远位置是无效的。

1、①②全部注释

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffc0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

2、①放开，②注释

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffd0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

3、①②全部放开

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffd0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

结论：该模式下，不会因为数组的内存访问越界报错，但似乎只能修改超过边界4个地址以外，修改位置4个地址以内的值是无效的。

1、①②全部注释

2、①放开，②注释

3、①②放开

```
[u2351520@oop ~]$ c++ -Wa  
[u2351520@oop ~]$ ./13-b7  
addr: 0xfffffc99d480  
0xfffffc99d470:fc99d4b0  
0xfffffc99d474:ffff  
0xfffffc99d478:bc714384  
0xfffffc99d47c:ffff  
0xfffffc99d480:0  
0xfffffc99d484:1  
0xfffffc99d488:2  
0xfffffc99d48c:3  
0xfffffc99d490:4  
0xfffffc99d494:5  
0xfffffc99d498:6  
0xfffffc99d49c:7  
0xfffffc99d4a0:8  
0xfffffc99d4a4:9  
0xfffffc99d4a8:0  
0xfffffc99d4ac:b  
0xfffffc99d4b0:0  
0xfffffc99d4b4:0  
0xfffffc99d4b8:e  
0xfffffc99d4bc:f
```

```
[u2351520@oop ~]$ c++ -Wa  
[u2351520@oop ~]$ ./13-b7  
addr: 0xfffff7f9f430  
0xfffff7f9f420:f7f9f460  
0xfffff7f9f424:ffff  
0xfffff7f9f428:b9bd4384  
0xfffff7f9f42c:ffff  
0xfffff7f9f430:0  
0xfffff7f9f434:1  
0xfffff7f9f438:2  
0xfffff7f9f43c:3  
0xfffff7f9f440:4  
0xfffff7f9f444:5  
0xfffff7f9f448:6  
0xfffff7f9f44c:7  
0xfffff7f9f450:8  
0xfffff7f9f454:9  
0xfffff7f9f458:a  
0xfffff7f9f45c:b  
0xfffff7f9f460:0  
0xfffff7f9f464:0  
0xfffff7f9f468:e  
0xfffff7f9f46c:f
```

```
[u2351520@oop ~]$ ./13-b7  
addr: 0xffffe189b370  
0xffffe189b360:e189b3a0  
0xffffe189b364:ffff  
0xffffe189b368:a6b14384  
0xffffe189b36c:ffff  
0xffffe189b370:0  
0xffffe189b374:1  
0xffffe189b378:2  
0xffffe189b37c:3  
0xffffe189b380:4  
0xffffe189b384:5  
0xffffe189b388:6  
0xffffe189b38c:7  
0xffffe189b390:8  
0xffffe189b394:9  
0xffffe189b398:cccccccc  
0xffffe189b39c:b  
0xffffe189b3a0:0  
0xffffe189b3a4:0  
0xffffe189b3a8:e  
0xffffe189b3ac:f
```

结论：Linux模式下，不会因为数组的内存访问越界报错。



§. 关于动态内存申请后越界访问的深度讨论

★ 如何判断普通数组的越界访问 (C++方式, 注意源程序后缀为.cpp)

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
using namespace std;
```

```
int main() {
    char a[10] = "0123456789";
    // *①*/ a[10] = 'a'; //此句越界
    a[14] = 'A'; //此句越界
    a[15] = 'B'; //此句越界
    // *②*/ a[10] = '\xcc'; //此句越界

    cout << "addr-a: " << static_cast<void*>(a) << endl;

    for (int i = -4; i < 16; i++) { // 注意, 只有0-9是合理范围, 其余
        都是越界读
        cout << static_cast<void*>(a + i) << ": " << hex <<
        static_cast<int>(a[i]) << endl;
    }

    return 0;
```

在理解P. 1/P. 2的情况下, 自行构造相似的程序, 来观察数组越界后的内存表现, 并验证与动态申请是否相似

要求:

- 2、数组用 `int a[10];` 形式
- 3、测试程序在下面五种环境下运行
VS2022 x86/Debug
VS2022 x86/Release
Dev 32bit-Debug
Dev 32bit-Release
Linux
- 4、每种讨论的结果可截图+文字说明, 如果几种环境的结果一致, 用一个环境的截图+文字说明即可(可加页)

VS2022 x86/Debug



1、①②全部注释

```
Microsoft Visual Studio 调试
addr:0075FB84
0075FB80:ffffffcc
0075FB81:ffffffcc
0075FB82:ffffffcc
0075FB83:ffffffcc
0075FB84:30
0075FB85:31
0075FB86:32
0075FB87:33
0075FB88:34
0075FB89:35
0075FB8A:36
0075FB8B:37
0075FB8C:38
0075FB8D:39
0075FB8E:ffffffcc
0075FB8F:ffffffcc
0075FB90:ffffffcc
0075FB91:ffffffcc
0075FB92:41
0075FB93:42
```

2、①放开，②注释

D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe

```
addr:00CFCC4
00CFCC0:ffffffcc
00CFCC1:ffffffcc
00CFCC2:ffffffcc
00CFCC3:ffffffcc
00CFCC4:30
00CFCC5:31
00CFCC6:32
00CFCC7:33
00CFCC8:34
00CFCC9:35
00CFCCA:36
00CFCCB:37
00CFCCC:38
00CFCCD:39
00CFCCF:ffffffcc
00CFCD0:ffffffcc
00CFCD1:ffffffcc
00CFCD2:41
00CFCD3:42
```

Microsoft Visual C++ Runtime Library

Debug Error!

Program: D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe

Module: D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe

File:

Run-Time Check Failure #2 - Stack around the variable 'a' was corrupted.

(Press Retry to debug the application)

中止(A) 重试(R) 忽略(I)

3、①②全部放开

```
Microsoft Visual Studio 调试
addr:008FF8E4
008FF8E0:ffffffcc
008FF8E1:ffffffcc
008FF8E2:ffffffcc
008FF8E3:ffffffcc
008FF8E4:30
008FF8E5:31
008FF8E6:32
008FF8E7:33
008FF8E8:34
008FF8E9:35
008FF8EA:36
008FF8EB:37
008FF8EC:38
008FF8ED:39
008FF8EE:ffffffcc
008FF8EF:ffffffcc
008FF8F0:ffffffcc
008FF8F1:ffffffcc
008FF8F2:41
008FF8F3:42
```

结论：1.VS2022的x86/Debug模式下，会判断数组内存访问是否越界。

2.该模式下，VS 会在动态分配的内存块的前后4个地址添加额外的特定值来判断访问是否越界。当这些值与特定值不符时，判断内存访问越界，报错；当值没有被改变，即使越界访问也不会判定错误。所以，当越界访问的位置超出特定值的范围，或者访问特定值的位置但并没有修改为不同的值时（情况3），程序并没有报错。

1、①②全部注释

```
Microsoft Visual Studio 调试
addr:00D7FB30
00D7FB2C:28
00D7FB2D:ffffffb2
00D7FB2E:2f
00D7FB2F:01
00D7FB30:30
00D7FB31:31
00D7FB32:32
00D7FB33:33
00D7FB34:34
00D7FB35:35
00D7FB36:36
00D7FB37:37
00D7FB38:38
00D7FB39:39
00D7FB3A:ffffffb9
00D7FB3B:76
00D7FB3C:03
00D7FB3D:13
00D7FB3E:ffffffd8
00D7FB3F:00
```

2、①放开，②注释

```
Microsoft Visual Studio
addr:00B3FBAC
00B3FBA8:28
00B3FBA9:ffffffb2
00B3FBAA:ffffffe3
00B3FBAB:00
00B3FBAC:30
00B3FBAD:31
00B3FBAE:32
00B3FBAF:33
00B3FBB0:34
00B3FBB1:35
00B3FBB2:36
00B3FBB3:37
00B3FBB4:38
00B3FBB5:39
00B3FBB6:61
00B3FBB7:76
00B3FBB8:05
00B3FBB9:13
00B3FBBA:05
00B3FBBB:00
```

3、①②全部放开

```
Microsoft Visual Studio 调试
addr:00D7FBBC
00D7FBB8:28
00D7FBB9:ffffffb2
00D7FBBA:0f
00D7FBBB:01
00D7FBBC:30
00D7FBBD:31
00D7FBBE:32
00D7FBBF:33
00D7FBC0:34
00D7FBC1:35
00D7FBC2:36
00D7FBC3:37
00D7FBC4:38
00D7FBC5:39
00D7FBC6:ffffffcc
00D7FBC7:76
00D7FBC8:05
00D7FBC9:13
00D7FBCA:ffffff97
00D7FBCB:00
```

结论：该模式下，不会因为数组的内存访问越界报错，但似乎只能修改超过边界4个地址以内，修改其他更远位置是无效的。

1、①②全部注释

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffc0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

2、①放开，②注释

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffd0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

3、①②全部放开

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffd0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

结论：该模式下，不会因为数组的内存访问越界报错，但似乎只能修改超过边界4个地址以外，修改位置4个地址以内的值是无效的。

1、①②全部注释

2、①放开，②注释

3、①②放开

```
[u2351520@oop ~]$ c++ -Wa  
[u2351520@oop ~]$ ./13-b7  
addr: 0xfffffc99d480  
0xfffffc99d470:fc99d4b0  
0xfffffc99d474:ffff  
0xfffffc99d478:bc714384  
0xfffffc99d47c:ffff  
0xfffffc99d480:0  
0xfffffc99d484:1  
0xfffffc99d488:2  
0xfffffc99d48c:3  
0xfffffc99d490:4  
0xfffffc99d494:5  
0xfffffc99d498:6  
0xfffffc99d49c:7  
0xfffffc99d4a0:8  
0xfffffc99d4a4:9  
0xfffffc99d4a8:0  
0xfffffc99d4ac:b  
0xfffffc99d4b0:0  
0xfffffc99d4b4:0  
0xfffffc99d4b8:e  
0xfffffc99d4bc:f
```

```
[u2351520@oop ~]$ c++ -Wa  
[u2351520@oop ~]$ ./13-b7  
addr: 0xfffff7f9f430  
0xfffff7f9f420:f7f9f460  
0xfffff7f9f424:ffff  
0xfffff7f9f428:b9bd4384  
0xfffff7f9f42c:ffff  
0xfffff7f9f430:0  
0xfffff7f9f434:1  
0xfffff7f9f438:2  
0xfffff7f9f43c:3  
0xfffff7f9f440:4  
0xfffff7f9f444:5  
0xfffff7f9f448:6  
0xfffff7f9f44c:7  
0xfffff7f9f450:8  
0xfffff7f9f454:9  
0xfffff7f9f458:a  
0xfffff7f9f45c:b  
0xfffff7f9f460:0  
0xfffff7f9f464:0  
0xfffff7f9f468:e  
0xfffff7f9f46c:f
```

```
[u2351520@oop ~]$ ./13-b7  
addr: 0xffffe189b370  
0xffffe189b360:e189b3a0  
0xffffe189b364:ffff  
0xffffe189b368:a6b14384  
0xffffe189b36c:ffff  
0xffffe189b370:0  
0xffffe189b374:1  
0xffffe189b378:2  
0xffffe189b37c:3  
0xffffe189b380:4  
0xffffe189b384:5  
0xffffe189b388:6  
0xffffe189b38c:7  
0xffffe189b390:8  
0xffffe189b394:9  
0xffffe189b398:cccccccc  
0xffffe189b39c:b  
0xffffe189b3a0:0  
0xffffe189b3a4:0  
0xffffe189b3a8:e  
0xffffe189b3ac:f
```

结论：Linux模式下，不会因为数组的内存访问越界报错。



§. 关于动态内存申请后越界访问的深度讨论

★ 如何判断普通数组的越界访问 (C++方式, 注意源程序后缀为.cpp)

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    int a[10] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    // *①*/ a[10] = 10; //此句越界
    a[14] = 14; //此句越界
    a[15] = 15; //此句越界
    // *②*/ a[10] = 0xcccccccc; //此句越界

    std::cout << "addr: " << a << std::endl;

    for (int i = -4; i < 16; i++) { // 注意, 只有0-9是合理范围,
        其余都是越界读
        std::cout << a + i << ": " << std::hex << a[i] << std::endl;
    }

    return 0;
}
```

在理解P. 1/P. 2的情况下, 自行构造相似的程序, 来观察数组越界后的内存表现, 并验证与动态申请是否相似

要求:

- 2、数组用 `int a[10];` 形式
- 3、测试程序在下面五种环境下运行
VS2022 x86/Debug
VS2022 x86/Release
Dev 32bit-Debug
Dev 32bit-Release
Linux
- 4、每种讨论的结果可截图+文字说明, 如果几种环境的结果一致, 用一个环境的截图+文字说明即可(可加页)

VS2022 x86/Debug



1、①②全部注释

```
Microsoft Visual Studio 调试
addr:0052FB6C
0052FB5C:cccccccc
0052FB60:ffffffffd
0052FB64:cccccccc
0052FB68:cccccccc
0052FB6C:00
0052FB70:01
0052FB74:02
0052FB78:03
0052FB7C:04
0052FB80:05
0052FB84:06
0052FB88:07
0052FB8C:08
0052FB90:09
0052FB94:cccccccc
0052FB98:7e1173e1
0052FB9C:52fbbc
0052FBA0:5d2083
0052FBA4:0e
0052FBA8:0f
```

2、①放开，②注释

```
D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe
addr:007DFD3C
007DFD2C:cccccccc
007DFD30:ffffffffd
007DFD34:cccccccc
007DFD38:cccccccc
007DFD3C:00
007DFD40:01
007DFD44:02
007DFD48:03
007DFD4C:04
007DFD50:05
007DFD54:06
007DFD58:07
007DFD5C:08
007DFD60:09
007DFD64:0a
007DFD68:e927f944
007DFD6C:7dfd8c
007DFD70:4c2083
007DFD74:0e
007DFD78:0f
```

Microsoft Visual C++ Runtime Library

Debug Error!

Program: D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe
Module: D:\desktop\oop\第六周 (国庆跳过第五周) \week6\Debug\13-b7.exe
File:

Run-Time Check Failure #2 - Stack around the variable 'a' was corrupted.

(Press Retry to debug the application)

中止(A) 重试(R) 忽略(I)

3、①②全部放开

```
Microsoft Visual Studio 调试
addr:004FFC78
004FFC68:cccccccc
004FFC6C:ffffffffd
004FFC70:cccccccc
004FFC74:cccccccc
004FFC78:00
004FFC7C:01
004FFC80:02
004FFC84:03
004FFC88:04
004FFC8C:05
004FFC90:06
004FFC94:07
004FFC98:08
004FFC9C:09
004FFCA0:cccccccc
004FFCA4:817f195f
004FFCA8:4ffcc8
004FFCAC:e62083
004FFCB0:0e
004FFCB4:0f
```

结论：1.VS2022的x86/Debug模式下，会判断数组内存访问是否越界。

2.该模式下，VS 会在动态分配的内存块的前后4个地址添加额外的特定值来判断访问是否越界。当这些值与特定值不符时，判断内存访问越界，报错；当值没有被改变，即使越界访问也不会判定错误。所以，当越界访问的位置超出特定值的范围，或者访问特定值的位置但并没有修改为不同的值时（情况3），程序并没有报错。

1、①②全部注释

```
Microsoft Visual Studio 调试
addr:0135F8FC
0135F8EC:682108
0135F8F0:682108
0135F8F4:15c04e8
0135F8F8:15bb228
0135F8FC:00
0135F900:01
0135F904:02
0135F908:03
0135F90C:04
0135F910:05
0135F914:06
0135F918:07
0135F91C:08
0135F920:09
0135F924:703cbe59
0135F928:135f970
0135F92C:681289
0135F930:01
0135F934:15bb228
0135F938:15c04e8
```

2、①放开，②注释

```
Microsoft Visual Studio 调试
addr:00F8FC68
00F8FC58:2e2108
00F8FC5C:2e2108
00F8FC60:12c04e8
00F8FC64:12bb228
00F8FC68:00
00F8FC6C:01
00F8FC70:02
00F8FC74:03
00F8FC78:04
00F8FC7C:05
00F8FC80:06
00F8FC84:07
00F8FC88:08
00F8FC8C:09
00F8FC90:0a
00F8FC94:c6e2138a
00F8FC98:f8fce0
00F8FC9C:2e1297
00F8FCA0:01
00F8FCA4:12bb228
```

3、①②全部放开

```
Microsoft Visual Studio 调试
addr:007BF8A4
007BF894:672108
007BF898:672108
007BF89C:a304e8
007BF8A0:a2b228
007BF8A4:00
007BF8A8:01
007BF8AC:02
007BF8B0:03
007BF8B4:04
007BF8B8:05
007BF8BC:06
007BF8C0:07
007BF8C4:08
007BF8C8:09
007BF8CC:cccccccc
007BF8D0:1f32328d
007BF8D4:7bf91c
007BF8D8:671297
007BF8DC:01
007BF8E0:a2b228
```

结论：该模式下，不会因为数组的内存访问越界报错，但似乎只能修改超过边界4个地址以内，修改其他更远位置是无效的。

1、①②全部注释

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffc0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

2、①放开，②注释

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffd0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

3、①②全部放开

```
D:\desktop\oop\第六周
addr:0065FEC2
0065FEBE:40
0065FEBF:00
0065FEC0:ffffffd0
0065FEC1:15
0065FEC2:30
0065FEC3:31
0065FEC4:32
0065FEC5:33
0065FEC6:34
0065FEC7:35
0065FEC8:36
0065FEC9:37
0065FECA:38
0065FECB:39
0065FECC:0a
0065FECD:00
0065FECE:00
0065FECF:00
0065FED0:61
0065FED1:62
```

结论：该模式下，不会因为数组的内存访问越界报错，但似乎只能修改超过边界4个地址以外，修改位置4个地址以内的值是无效的。

1、①②全部注释

2、①放开，②注释

3、①②放开

```
[u2351520@oop ~]$ c++ -Wa  
[u2351520@oop ~]$ ./13-b7  
addr: 0xfffffc99d480  
0xfffffc99d470:fc99d4b0  
0xfffffc99d474:ffff  
0xfffffc99d478:bc714384  
0xfffffc99d47c:ffff  
0xfffffc99d480:0  
0xfffffc99d484:1  
0xfffffc99d488:2  
0xfffffc99d48c:3  
0xfffffc99d490:4  
0xfffffc99d494:5  
0xfffffc99d498:6  
0xfffffc99d49c:7  
0xfffffc99d4a0:8  
0xfffffc99d4a4:9  
0xfffffc99d4a8:0  
0xfffffc99d4ac:b  
0xfffffc99d4b0:0  
0xfffffc99d4b4:0  
0xfffffc99d4b8:e  
0xfffffc99d4bc:f
```

```
[u2351520@oop ~]$ c++ -Wa  
[u2351520@oop ~]$ ./13-b7  
addr: 0xfffff7f9f430  
0xfffff7f9f420:f7f9f460  
0xfffff7f9f424:ffff  
0xfffff7f9f428:b9bd4384  
0xfffff7f9f42c:ffff  
0xfffff7f9f430:0  
0xfffff7f9f434:1  
0xfffff7f9f438:2  
0xfffff7f9f43c:3  
0xfffff7f9f440:4  
0xfffff7f9f444:5  
0xfffff7f9f448:6  
0xfffff7f9f44c:7  
0xfffff7f9f450:8  
0xfffff7f9f454:9  
0xfffff7f9f458:a  
0xfffff7f9f45c:b  
0xfffff7f9f460:0  
0xfffff7f9f464:0  
0xfffff7f9f468:e  
0xfffff7f9f46c:f
```

```
[u2351520@oop ~]$ ./13-b7  
addr: 0xffffe189b370  
0xffffe189b360:e189b3a0  
0xffffe189b364:ffff  
0xffffe189b368:a6b14384  
0xffffe189b36c:ffff  
0xffffe189b370:0  
0xffffe189b374:1  
0xffffe189b378:2  
0xffffe189b37c:3  
0xffffe189b380:4  
0xffffe189b384:5  
0xffffe189b388:6  
0xffffe189b38c:7  
0xffffe189b390:8  
0xffffe189b394:9  
0xffffe189b398:cccccccc  
0xffffe189b39c:b  
0xffffe189b3a0:0  
0xffffe189b3a4:0  
0xffffe189b3a8:e  
0xffffe189b3ac:f
```

结论：Linux模式下，不会因为数组的内存访问越界报错。



§. 关于动态内存申请后越界访问的深度讨论

★ 最后一页：仔细总结本作业（多种形式的测试程序/多个编译器环境/不同结论），谈谈你对内存越界访问的整体理解包括但不限于操作系统/编译器如何防范越界、你应该养成怎样的使用习惯来尽量防范越界

总结：

部分操作系统或编译器有内存保护机制，防止程序访问未分配的内存区域。当程序尝试访问不属于它的内存时，操作系统会抛出异常，终止程序。这种机制有效地防止了某些类型的内存越界访问。在VS2022的x86/Debug模式下，编译器会在动态分配的内存块前后添加特定值，以检测越界访问。当访问这些特定值并且修改了它们时，程序会报错。而在其他模式下以及Linux环境下，编译器通常不会添加这些保护机制，因此越界访问不会引发错误，导致潜在的严重后果。

为了尽量防范内存越界访问，我们在编写代码时要了解各种操作系统和编译器在防范上采取的措施，机制的工作原理；同时要养成良好的编程习惯，以减少内存越界带来的风险。通过合理的内存管理、边界检查及使用合适的工具，可以有效预防和检测内存越界问题，提高程序的安全性和稳定性。